

Experiment:12: Creation of Binary Search Tree (BST)

Aim: To write a program to create a Binary Search Tree (BST) and perform inorder traversal.

Theory: A Binary Search Tree (BST) is a non-linear data structure in which each node has at most two children:

- Left child contains values less than the parent node
- Right child contains values greater than the parent node

BST is mainly used for:

- Fast searching
- Sorting data
- Efficient insertion and deletion

Inorder Traversal of BST gives elements in sorted order.

Algorithm:

1. Start
2. Create a node with given value
3. If tree is empty, make it root
4. If value < root → insert in left subtree
5. If value > root → insert in right subtree
6. Repeat until all nodes are inserted
7. Perform inorder traversal (Left → Root → Right)
8. Display elements
9. Stop

Code:

```
#include <stdio.h>
#include <stdlib.h>
```

// Structure of node

```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
```

// Create new node

```
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
```

```

return newNode;
}

// Insert node in BST
struct Node* insert(struct Node* root, int value) {
    if (root == NULL)
        return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

```

```

// Inorder traversal
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

```

```

// Main function
int main() {
    struct Node* root = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter values:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("Inorder Traversal: ");
    inorder(root);

    return 0;
}

```

Sample Input:

Enter number of nodes: 5
Enter values: 50 30 70 20 40

Output:

Inorder Traversal: 20 30 40 50 70

Result:

The Binary Search Tree was successfully created and inorder traversal displayed elements in sorted order.